

STRATUS

Clustered hierarchical memory for scientific agents

Open components · self-hosted · built to scale from one agent to thousands

Apache-2.0 · open stack

OVERVIEW

A memory store shaped like memory

Agents need more than a chat log. STRATUS layers memory into four strata — from raw observation to long-lived persona — so recall returns the right grain of context for the question.

T0 • Stream

Raw conversation
and observation log.
Every turn, every
event.

T1 • Atoms

Distilled atomic
memories — facts,
preferences, rules.

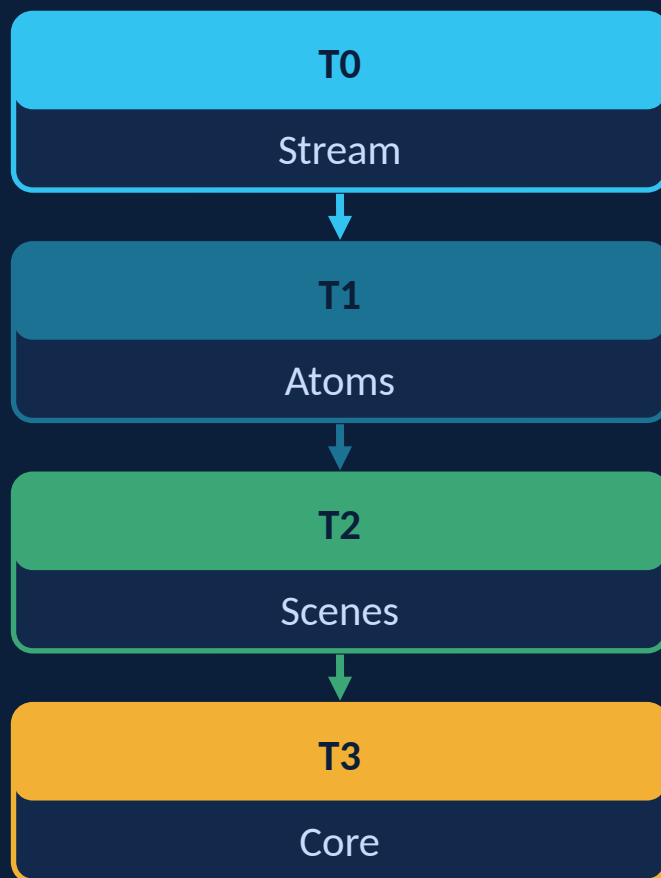
T2 • Scenes

Synthesized episodic
blocks summarizing
what happened.

T3 • Core

Long-lived persona
and project core. The
durable self.

Four tiers, one open backing store



BACKING STORE (all open)

SQLite + sqlite-vec

Dense vector recall — cosine similarity over per-tier

SQLite FTS5

Lexical recall — BM25 full-text over the same content.

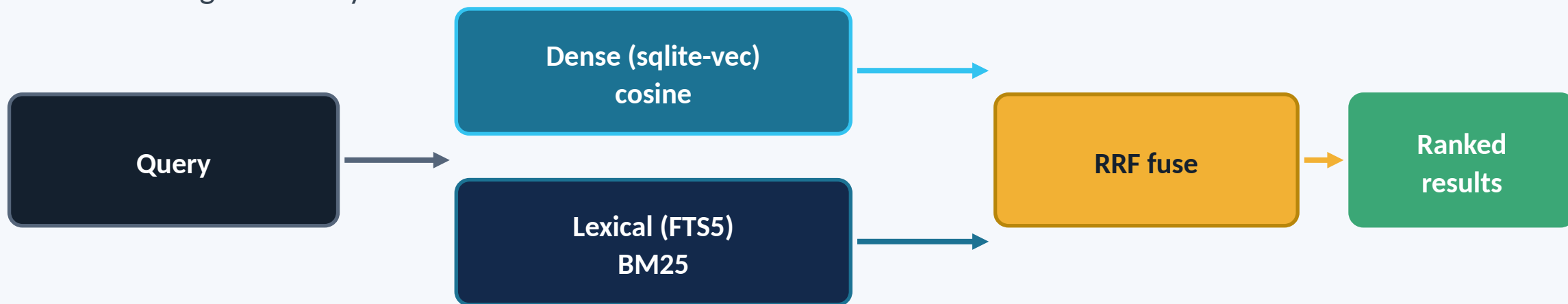
Local filesystem

Scene (T2) and core (T3) blobs, beside the database.

No proprietary database. No managed service. Runs fully offline.

Dense + lexical, fused

A query runs against both indexes; results merge by reciprocal-rank fusion. Semantic matches and exact-term matches each get their say — neither dominates.



One process, one box, 1–50 agents

`stratus serve` embeds an entire store in a single Node process over one SQLite database. Install, smoke-test, and a live gateway in under a minute — the right shape for a workstation or a single VM.

Install

```
./install.sh
```

- checks Node ≥ 20
- builds + smoke-tests
- links the CLI

Run

```
stratus serve
stratus health
stratus smoke
```

Offline by default.

Use

```
POST /stream/add
POST /atoms/upsert
POST /*/search
POST /core/read|write
```

THE QUESTION

What does it take to run thousands of agents?

Design point: 1,000–10,000 agents, ~20% active at once.

2,000

active agents (20% of 10k)

~2k/s

sustained writes

**~600/
s**

recalls (embed+search)

**10s
of M**

vectors in aggregate

Two numbers dominate the plan: embedding throughput, and vector-search QPS × index size.

Four moves, all on the open stack

1 • Store pool + sharding

One SQLite DB per agent, lazily opened and LRU-evicted. Agents map to shards by `hash(agent_id)`. Removes write contention entirely.

2 • Stateless gateway fleet

K replicas behind a consistent-hash router, shared-nothing — each owns a disjoint slice of agents. The biggest unlock, no shared hot-path state.

3 • Dedicated embedding tier

Batched GPU embedding service (vLLM/Ollama, open model) off the request path. Decouples embed throughput from gateway count.

4 • Async pipeline workers

T0→T1→T2 distillation runs on a worker pool off the hot path, reusing the lab's shared open-model LLM tier.

ROLLOUT

Five phases, each independently shippable

Phase 1

Store pool + sharding

→ 100s of agents on one node

Phase 2

Gateway fleet + router

→ 1,000s across a few nodes

Phase 3

Batched embedding tier

→ removes write-path bottleneck

Phase 4

Async pipeline workers

→ memory quality without blocking

Phase 5

ANN + aggregation + federation

→ largest agents, multi-collection

Pilot — up to ~250 agents

Single node. One workstation or one cloud VM.

CPU

16 cores

gateway event loop + SQLite + FTS across active stores

RAM

64 GB

hot store pool (LRU) + FTS/vec page cache

Disk

2 TB NVMe

per-agent SQLite + vectors; NVMe for random I/O

GPU

1 × 24 GB

batched embedding model (A10 / L4 / 3090 class)

Net

10 GbE

local — not a constraint at this size

Department — ~1,000–2,500 agents

Shared-nothing shards; turn the embedding-GPU knob first when p99 climbs.

Gateways

3–4 nodes

16-core / 64 GB / 2 TB NVMe each, agent→node by hash

Router

1 small node

consistent-hash routing (or HW load balancer)

Embed GPU

1–2 × 40–48 GB

A100-40 / L40S — ~600–1,500 embeds/sec batched

Workers

8–16 CPU

async distillation; LLM calls go to shared tier

Storage

~8 TB NVMe

per-agent DBs + blobs, 3-copy durability

Lab-wide — ~10,000 agents

Binding constraint is embedding-GPU throughput, then aggregate NVMe IOPS.

Gateways

10–16 nodes

32-core / 128 GB / 4 TB NVMe — wider shard fan-out

Router

HA pair

consistent hashing, no single point of failure

Embed GPU

4–8 × 80 GB

A100-80 / H100 / MI300 — 10k+ embeds/sec

Workers

40–80

keep distillation backlog near zero

Storage

40–80 TB NVMe

tens of millions of vectors + blobs, ≥3× replicated

SIZING RULES

Plan GPU first, fast disk second, CPU last

Embedding GPUs

$\text{ceil}(\text{peak_embeds/sec} \div \text{per-GPU batched throughput})$. One 40–80 GB GPU $\approx$ 1–3k embeds/sec. Size with $\geq 25\%$ headroom.

Gateway nodes

$\text{ceil}(N_{\text{agents}} \div \text{agents_per_node})$, where $\text{agents_per_node} \approx 250\text{--}700$ by activity and RAM for the hot pool.

NVMe storage

$N_{\text{agents}} \times \text{per_agent_bytes} \times \text{replication}$. Always NVMe — SQLite + vector queries are random-I/O bound.

RAM

working-set stores $\times$ per-store page cache. Cold agents cost only disk, not memory.

What we explicitly do not need

- ✓ No proprietary vector database — sqlite-vec + optional faiss/hnsw covers it.
- ✓ No managed cloud queue, cache, or warehouse — a plain work queue + per-agent SQLite fan-out suffices.
- ✓ No dedicated distillation cluster — reuse the lab's shared open-model inference tier.
- ✓ No shared mutable cross-node state on the hot path — sharding by agent keeps the fleet shared-nothing.

One agent to thousands — four moves, one open stack

Store pool + sharding · stateless gateway fleet · batched embedding tier · async pipeline workers.

Dominated by embedding GPUs and fast NVMe — gateway CPU is the cheap part. Three-copy durability throughout.

STRATUS · open · self-hosted · Apache-2.0